AESC v2: Two-Speed Memory Architecture (Revised Technical Report, v3.1

)

Oct 11, 2026 · @Giridhari

Abstract

AESC (Adaptive Event-State Computation) is a recurrent language-model architecture proposed by the author, Giridhari Karmakar. It combines a k-step neural field, a predictive dendrite, top-k skill routing and two-speed memory: fast weights written token by token by a surprise-gated delta rule, and slow weights trained by backpropagation and replay.

The author's hypothesis is that a model that carries a state and a fast-weight matrix across tokens can store and retrieve new associations while it reads. In a 9.92M-parameter reference implementation, the fast memory solved a 4-pair associative-recall task at 1.000 accuracy after 800 steps (chance 0.125). The first 10M implementation scored 0.37 with fast updates on or off, and four of its modules received zero gradient.

This revision describes eleven failure modes found in the first implementation and the changes made for them, audits each component in turn, and adds a safety observability design. Only the fast memory has experimental support so far; the other blocks are unproven.

Author's note

This report describes a research hypothesis and an architecture that I propose. It does not claim that the architecture works in general, and it does not claim that it will work as intended.

What I did. I built a 9.92M-parameter reference implementation and tested it on one small synthetic task, associative recall with 4 key-value pairs. In that run it reached accuracy 1.000. I also audited my first implementation, found eleven failures (Section 9), and revised the design. The component audit in Section 5 and the diagrams were added later. The audit was done from the report and its equations, and I have not yet checked it against the code.

What I do not know. The results are single-seed runs of a small model on a CPU, on one toy task. I do not know whether the slow memory, neuromodulator, metacognition, router or predictor help at all, because that task does not need them. I do not know how the model behaves on harder tasks, at a larger scale, or against baselines over several seeds.

Where I may be wrong. In my own understanding the design is correct, but that is a belief, not a result. I may be wrong in several places:

- the one-step delay that is meant to prevent label leakage (its test has not been run)
- the predictor target and the risk of embedding collapse
- the router, which may compute every skill and so save no compute
- the slow memory, whose read and write may not use the same key space
- the rule-based controller, which may add nothing
- the safety design, which is not implemented

What I will do. I did my best on this version. If testing shows a problem, I will try to fix it. I will report the problem and the fix in the next revision, and I will keep failed results in the report instead of removing them. A block that shows no benefit on the task built for it will be reported as "no evidence yet", or removed.

1. Hypothesis and proposed architecture

The hypothesis and the architecture in this report are the author's own. Both are proposals; only a small part of them has been tested.

Hypothesis. A small recurrent model with test-time memory can store and retrieve new associations while it reads. It does so because it carries a state and a fast-weight matrix across tokens, where a transformer has no built-in short-term plasticity. The hypothesis is untested beyond one synthetic task.

Claims. The report states two component-level claims that the experiments in Section 8 can test:

1. C1. A fast memory with a differentiable delta-rule write solves associative recall. Supported: accuracy 1.000 on 4-pair recall (Section 8, Run 1).
2. C2. Each added block (slow memory, neuromodulator, metacognition, router, predictor) beats a control with the same skills and without that block, on a task built to need it. Untested.

Proposed architecture. One recurrent state per token. A dendrite reads a local window, a neural field iterates k steps, a router picks skills, and a fast memory is written by a local delta rule while a slow memory is trained by backpropagation and replay. Section 3 draws it and Appendix A gives the equations.

Contributions of this revision.

- A revised design and equations (Section 3, Appendix A)
- A gradient policy for the fast write, backed by a measured ablation (Section 4)
- A component-by-component audit with fixes and first tests (Section 5)
- Results from a reference implementation and an audit of the first one (Section 8)
- A safety observability design (Section 7)

2. Related work

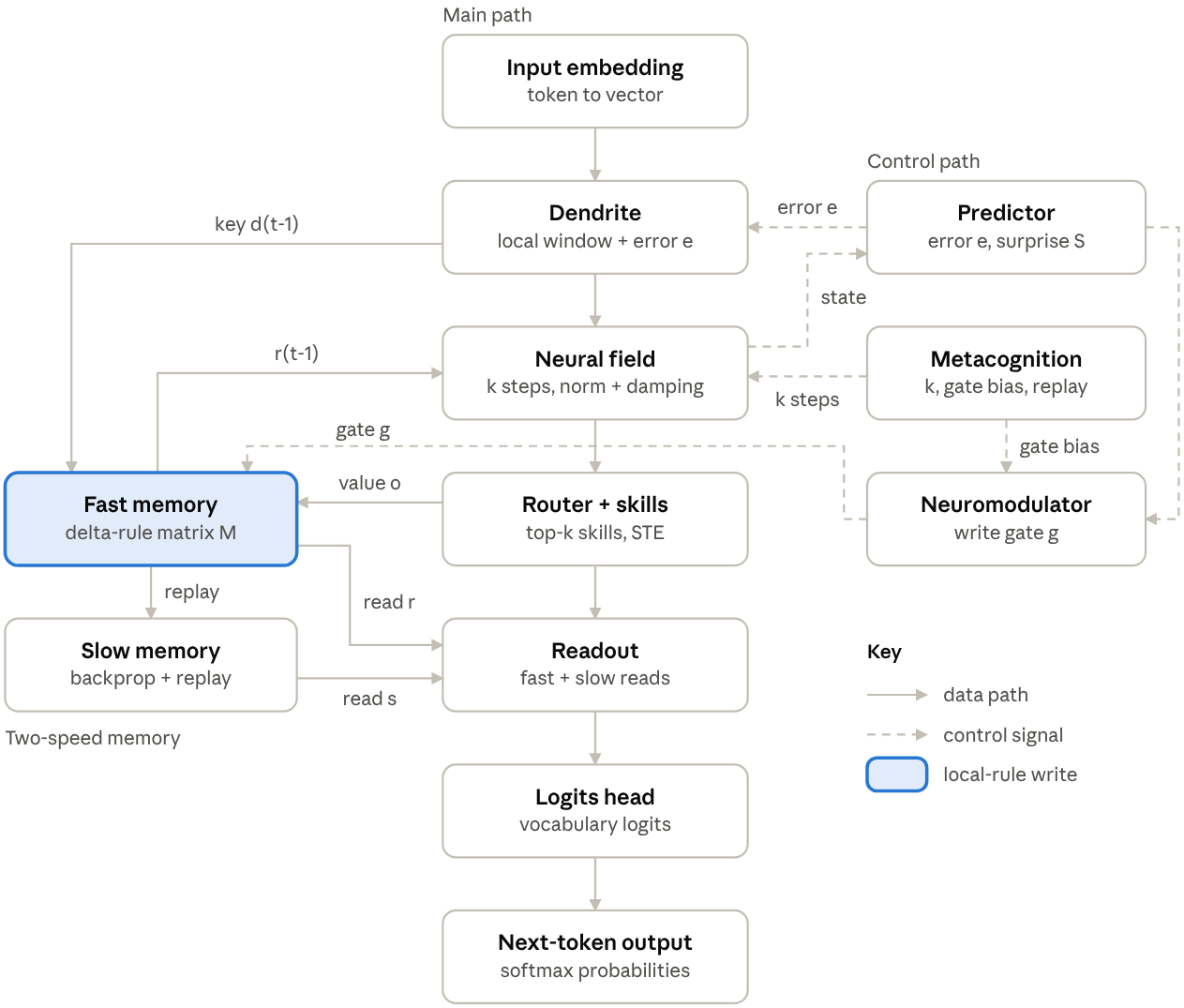
Each component has precedent; the contribution is the combination and the audit signals, not any single part.

- **Fast weights.** Schmidhuber (1992) and Ba et al. (2016) use a fast matrix that changes within a sequence. Schlag et al. (2021) show that linear attention is a fast-weight programmer, and DeltaNet (Yang et al., 2024) uses the delta rule that AESC's write also uses.
- **Test-time memory.** Titans (Behrouz et al., 2024) writes memory driven by surprise, which is close to the AESC gate.
- **State-space models.** Mamba (Gu and Dao, 2023) is the main recurrent baseline for the comparisons in Section 8.
- **Predictive coding.** Rao and Ballard (1999) motivate the dendrite error signal.
- **Two-speed memory.** McClelland, McNaughton and O'Reilly (1995) motivate fast learning plus replay into slow weights.
- **Looped computation.** Universal Transformers (Dehghani et al., 2018) reuse weights over steps, as the k-step field does.
- **Routing.** Sparse mixtures of experts (Shazeer et al., 2017) and straight-through estimators (Bengio et al., 2013) support the router.

3. Proposed architecture

Each token passes through the blocks below in a fixed order, and only the fast memory is written by a local rule. The figure shows the data path, the control path and the two-speed memory.

Only the fast memory is written by a local rule



architecture · main path, control path, two-speed memory

Tokens move down the center column. Dashed arrows are control signals, and surprise S runs from the predictor to the neuromodulator. The highlighted fast memory is the only block written by a local rule. This figure and the table below show the reference form, which runs token by token. The parallel form is described later in this section.

Per-token loop. The outer loop runs once per token; the inner loop runs the field k_t times. The dendrite output d_t and the memory read r_{t-1} stay fixed inside the inner loop, and the field state h carries over from token to token.

```
h, M, r, e = init                       # once per sequence
for t in tokens:
    x_t = Embed(token_t)
    d_t = Dendrite(x_t, x_{t-1}, ..., e_t)
    S_t, g_t, k_t = Control(e_t, h)
    for _ in range(k_t):                # k-step neural field
        h = Field(h, d_t, r_{t-1})
    o_t = RouterSkills(h)
    M   = Write(M, d_{t-1}, o_t, g_t)  # fast memory, delta rule
    r_t = Read(M, d_t)
    z_t = Readout(o_t, r_t, u_t)       # u_t: slow-memory read
    x_hat_{t+1} = Predictor(h)         # e_{t+1} is formed when x_{t+1}
arrives
```

The slow-memory read u_t goes to the readout only. The field receives only the fast-memory read r_{t-1}. Proposed changes to this forward pass are listed one by one in Appendix A.2 and are untested.

Block	Role	Learning
Input embedding	Token to vector	Backprop
Dendrite	Local causal window; adds the previous step's prediction error	Backprop (next-token loss + error energy)
Neural field	k-step recurrent state with norm and damping; reads memory from the previous step	Backprop
Predictor, compare, surprise	Predicts the next input; surprise is the mean absolute error, detached	Backprop (error energy)
Neuromodulator	Write gate g from relative surprise and state activity	Gradient through the write (Section 4)
Router and skills	Top-k skills; learned weights kept in the forward pass	Backprop + straight-through

Block	Role	Learning
Fast memory	Per-sequence matrix, delta-rule write scaled by g, learned decay	Local rule in form; gradient to key, value and gate inside a truncated window
Slow memory	Consolidated weights read by a query	Backprop + replay
Metacognition	Chooses k, gate bias and replay timing from surprise statistics	Rules (a learned policy is future work)
Readout and logits head	Adds fast and slow reads to the skill output; vocabulary logits, then softmax gives the next-token output	Backprop

Timing rule (intended to avoid leakage). The predictor forecasts x(t+1) from the state after token t. The error is computed when x(t+1) arrives and is used from step t+1 on. Nothing computed from x(t+1) may reach the logits that predict x(t+1).

Memory addressing. The write key comes from the previous step's dendrite feature, and the write value comes from the current skill output. The read query comes from the current dendrite feature, so a stored pair is retrieved when its key token appears again.

Notation note. Appendix A uses k for three things: field steps k_t, the number of routed skills, and the memory key k_t. Renaming them (K_field, K_route, key_t) is planned.

Parallel form (proposed): whole sequence at once

The reference model runs one token at a time. This subsection describes a proposed form that processes the whole sequence in one forward pass, as a transformer does in training, and runs token by token only at inference. It changes what the model computes, so none of the reference results apply to it until the tests below are run.

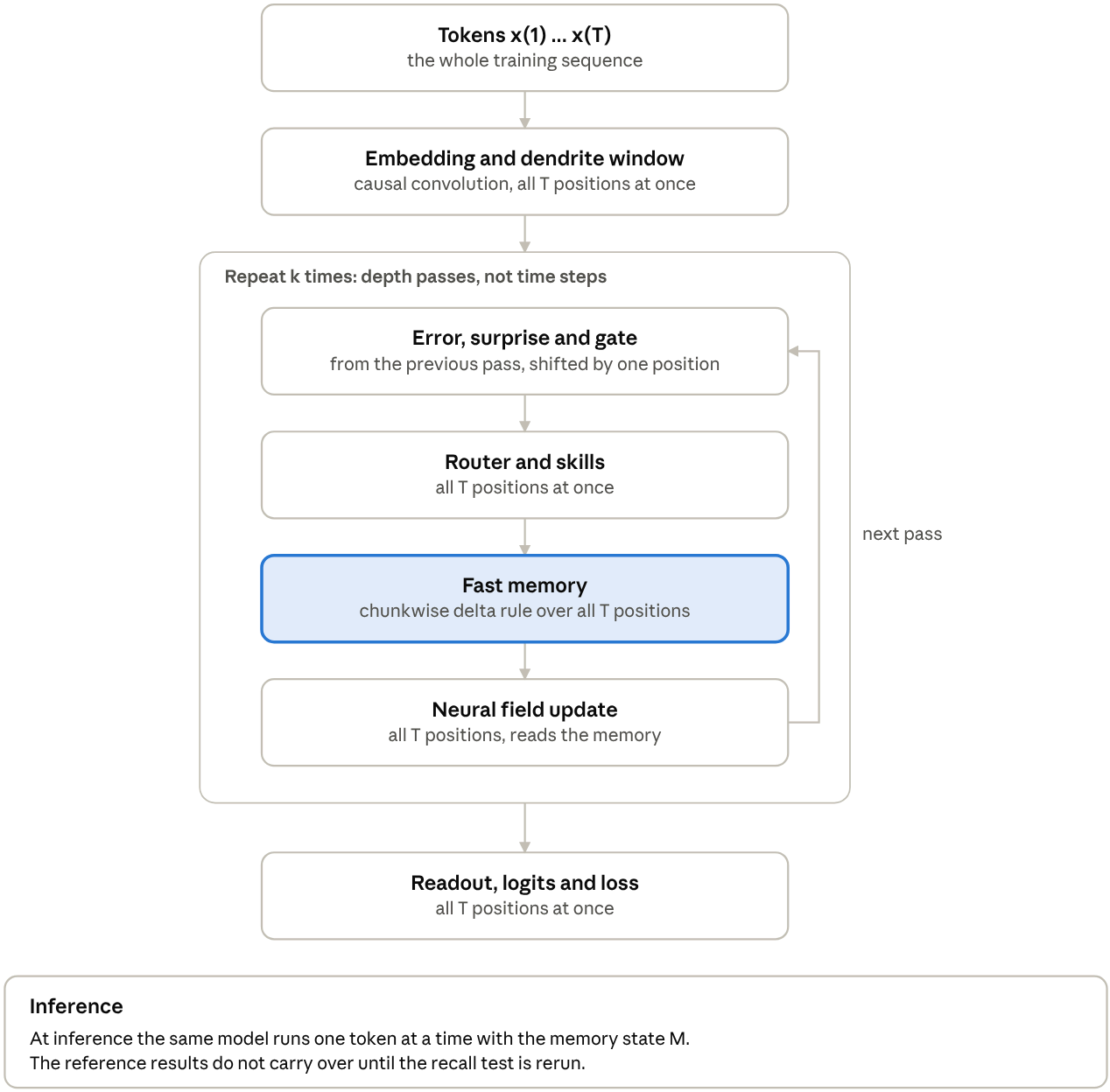
What changes.

- The time recurrence moves into the fast memory. For fixed k, v and g the delta-rule write is linear in M, so it can be computed for all positions at once with a chunkwise algorithm in the spirit of DeltaNet (Yang et al., 2024). The same equations run as a one-token recurrence at inference.

- The neural field stops being a recurrence over time. Its k steps become k passes in depth over all positions, and the memory read is its only link to earlier tokens. The state h(t) no longer depends on h(t-1).
- The predictor and the error use the states of the previous pass, shifted by one position, so the error still arrives one step late.
- The running surprise average is a causal moving average, which is a linear scan.
- Router, skills, slow read, readout, logits and loss already work per position, so they run over all T positions at once.

```
X = Embed(tokens)                                # all T positions
D_win = DendriteWindow(X)                        # causal convolution
H = Field0(D_win)                                # token-local start state
for j in 1..k:                                   # depth passes, not time steps
    E = X - shift(Predictor(H))                  # E[t+1] = x(t+1) - f_p(H[t]),
one step late
    S = Surprise(E); G = Gate(S, causal_average(S), H)
    D = D_win + gamma * E
    O = RouterSkills(H)                          # all T positions at once
    M, R = DeltaMemory(key=shift(D), value=O, gate=G, query=D)   # chunkwise
    H = FieldStep(H, D, shift(R))                # reads the memory read of the
previous position
O = RouterSkills(H)
Z = Readout(O, R, SlowRead(D)); loss = CE(Z, shifted tokens)     # all T
positions
```

Proposed parallel form: all positions at once, k passes in depth

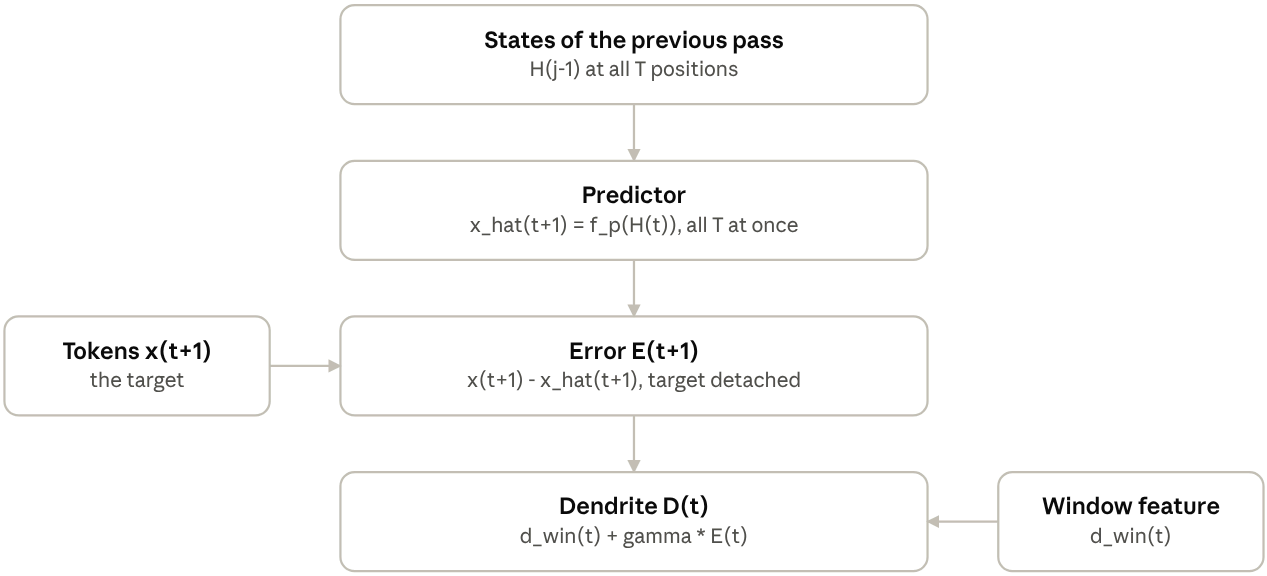


proposed parallel form · whole sequence, k depth passes

All T positions run together, and only the k depth passes are sequential. At inference the same memory runs as a recurrence, one token at a time.

Parallel form: dendrite and error.

Parallel form: the error comes from the previous pass

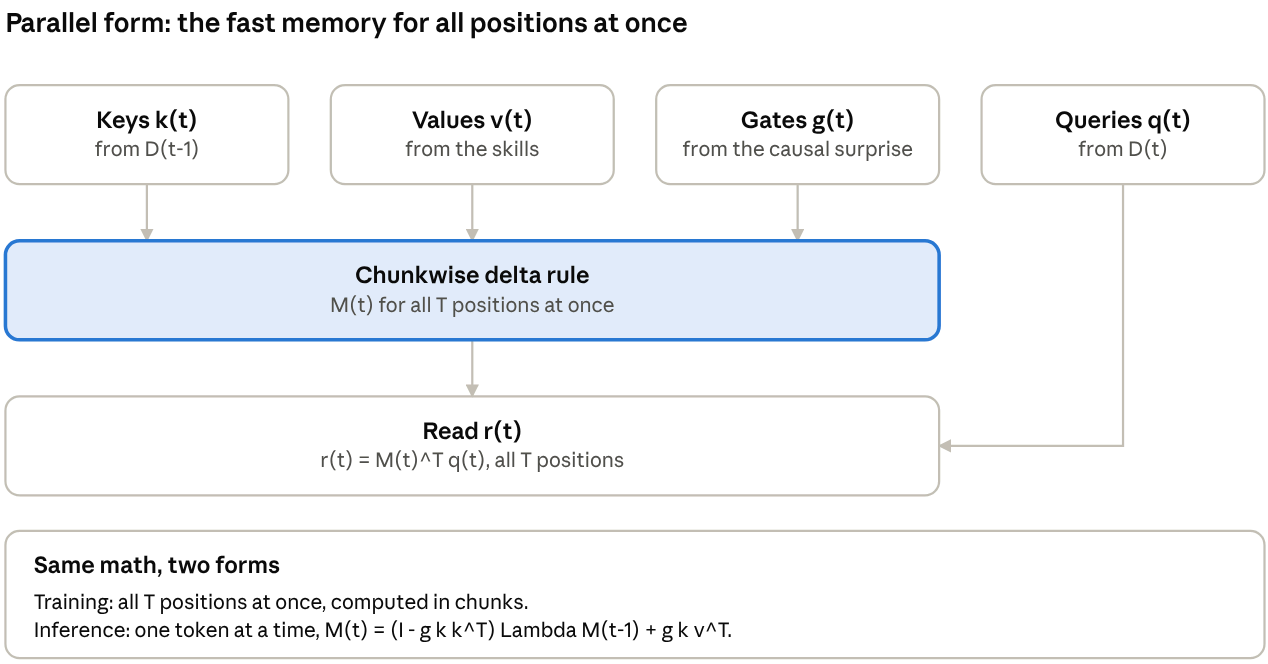


Leakage rule
E(t) uses x(t) and the previous pass at position t-1, so position t is built not to see x(t+1).
The leakage test of Section 3 checks this; it has not been run.

parallel form · error from the previous pass

The predictor runs on the states of the previous pass at all positions, and the result is shifted by one position. The error therefore still arrives one step late, and the target is detached (Change 1 of Appendix A.2).

Parallel form: fast memory.



parallel form · fast memory over all positions

Keys, values, gates and queries are all known for the whole sequence before the memory is computed. The memory is linear in M for fixed k, v and g, which is what lets it be computed in chunks. The equivalence test of this section checks that the chunked and the one-token forms give the same logits.

Risks.

- The field loses its nonlinear recurrence over time. Whatever that recurrence contributed, if anything, is gone, and the field becomes a depth refinement. Whether this hurts is unknown.
- The chunkwise algorithm is known for the delta rule with a scalar gate. The diagonal decay Lambda and the per-token gate g(t) of this model have to be checked, with a scalar decay as the fallback.
- Adaptive k per token is dropped in this form, because it cannot be batched cheaply. One k is used per batch.
- Router and skills stay dense: every skill is computed in every pass. Each pass also keeps its own fast memory, so the state carried at inference is k matrices instead of one, although the weights are shared.

Tests before any claim.

1. Equivalence: the whole-sequence pass and the one-token recurrence give the same logits within numerical error.
2. Leakage: change x(t+1) and check that the logits at step t do not change.
3. Recall: 4 pairs at 0.95 or higher, then 8 to 64 pairs with distractors.
4. Speed: tokens per second against the reference form and the baselines on the same hardware, with the gradient check of Appendix B item 6 repeated.

Open design problems and proposed fixes

These five problems were found by reading the parallel form against its own equations. The fixes are proposals, and none is implemented or tested. Each fix has a test, and a fix that fails its test is dropped and reported. The equations are in Appendix A.4.

1. **A state of k matrices and a k-fold cost.** Fix: write the memory in pass 1 only, and read it in every pass. The state at inference is then one matrix per head instead of k. The skills run twice, in pass 1 for the values and at the end for the readout, instead of k times, because passes 2 to k only refine the field, which has about 2.6K parameters in the reference model. One open point: reading the memory with new queries in later passes needs the memory at every position, so the chunkwise algorithm has to produce those reads without storing every M(t). Test: recall and loss stay within the seed range of the version with one memory per pass, and FLOPs per token and state size are reported.
2. **Pass 1 has no sequence context.** Fix: a context channel in the start state, a diagonal linear recurrence over time that is computed by a parallel scan, so that keys, values and surprise in pass 1 already carry earlier context. Test: with and without the channel, on recall and on a task whose answer depends on an earlier token.
3. **No nonlinear recurrence over time.** The context channel restores a linear one. Whether a nonlinear one is needed is tested directly: on a state-tracking task such as the parity of a bit stream, compare the reference form with the parallel form. If the parallel form fails where the reference form succeeds, the nonlinear recurrence matters and a hybrid has to be considered.
4. **The chunkwise algorithm and the diagonal decay.** Fix: use a scalar decay lambda per head instead of the diagonal Lambda over key dimensions. Chunkwise algorithms for the delta rule with a scalar decay exist in published work, and the exact reference has to be checked before this report cites it. Per-key forgetting is recovered by using several heads, each with its own scalar decay. Test: the equivalence test of this section, and scalar against diagonal decay in the reference form on recall and overwrite tasks.

5. **Memory capacity.** A d_k by d_v matrix can hold about d_k independent pairs, so 8 to 64 pairs may interfere at d_k = 64. This is a rule of thumb, not a measured limit. Fix: several heads, each with its own memory and decay, so that the capacity grows with the number of heads, and a larger d_k if the capacity curve falls early. Test: recall with 4, 8, 16, 32 and 64 pairs for 1, 2 and 4 heads, over three seeds, reporting where accuracy drops.

The figure of the parallel form above shows the version of Appendix A.3. With fix 1, the fast memory is written in pass 1 only and read in every pass.

Working flows

Three flows show how the architecture runs: one token, one training step, and the safety monitor.

One token.

One token: nine steps, with the field repeated k(t) times

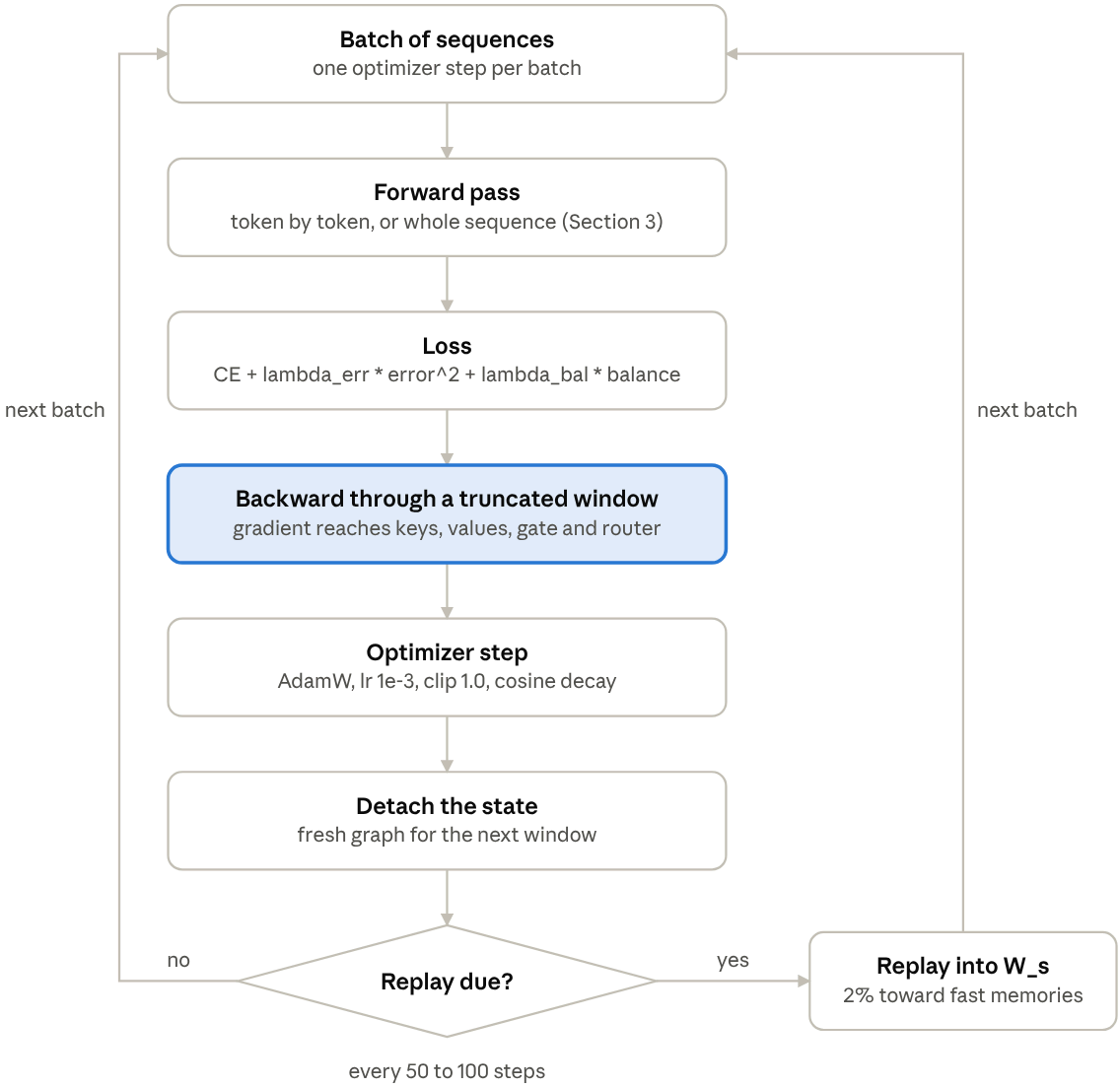


per-token flow · nine steps, one inner loop

In the reference form, each token runs the nine steps in order. The field step repeats k(t) times, and the predictor output waits for the next token before it becomes an error.

One training step.

One training step: the write stays in the graph, replay runs every 50 to 100 steps

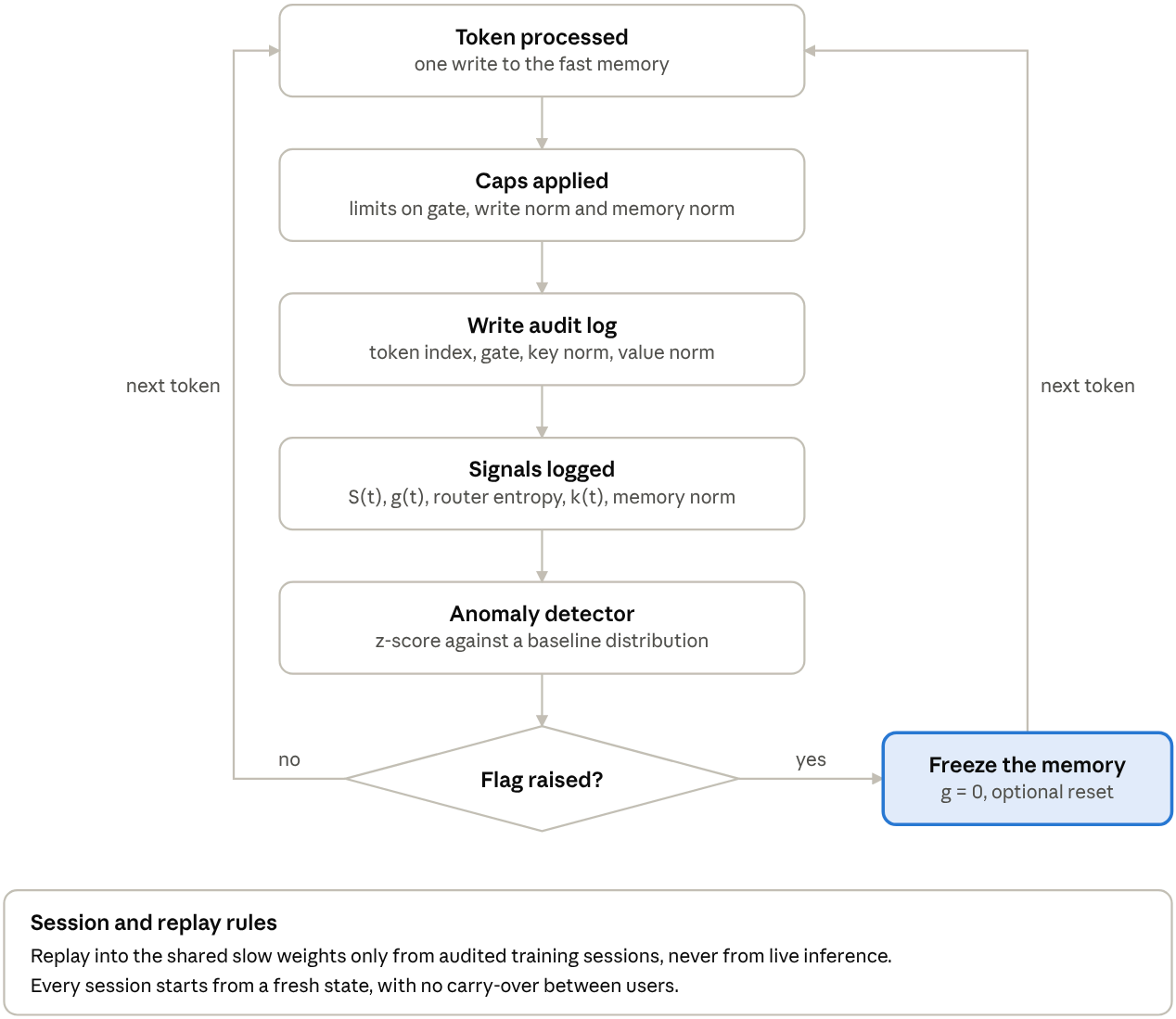


training step flow · one decision

The highlighted step is the gradient policy of Section 4: the delta-rule write stays inside the graph for a truncated window, and the state is detached between windows.

Safety monitor.

A flagged write freezes the memory, and replay never comes from live sessions



safety monitor flow · one decision

The monitor runs on every token. A raised flag sets the gate to zero so the memory only decays. This is a design; it has not been implemented or evaluated (Section 7).

4. Training procedure

Training takes one optimizer step per batch on the next-token loss, and the fast memory is written at every token. The write stays inside the gradient graph for the key map, the value map and the gate.

```
L = \sum_t \mathrm{CE}(z_t, y_t) + \lambda_{err}\,\lVert e_t \rVert^2 +
\lambda_{bal}\, B(\pi_t)
```

B is a router balance term that replaces the entropy term of v1. The reference implementation uses lambda_err = 0.1 and lambda_bal = 0.01.

Gradient policy for the fast write. v1 (Proposition 3) put a stop-gradient on the write, which cuts all gradient to W_k, W_v and the gate. In the reference implementation a stop-gradient write gave recall 0.75 against 1.000 with the write inside the graph, and the neuromodulator gradient was exactly zero. The policy is therefore to keep the delta-rule write differentiable inside a truncated window and to detach the state between windows. Purely local credit assignment stays a research direction; if it is pursued, the neuromodulator needs its own auxiliary loss, for example a gate target based on how much the write lowers the next-token loss. That loss is a proposal and has not been tested.

Routing. The forward pass uses the softmax over the top-k logits, so unequal weights survive. The backward pass uses the gradient of the full softmax (straight-through). In a unit test, hard top-k gave router gradient to only the 2 picked skills out of 4, and straight-through gave it to all 4.

Replay and optimizer. Every 50 to 100 steps, Metacognition triggers replay: the slow weights move 2% toward the surprise-weighted average of the fast memories in the batch. The reference notebook uses AdamW with learning rate 1e-3, 100 warmup steps, cosine decay to 10%, and gradient clipping at 1.0. The 1e-4 to 3e-4 range in v1 was not tested. A curriculum of structured tasks first is still recommended and still untested.

Changes proposed by the audit (Section 5), not yet tested.

- Detach the target in the error: e = sg(x_{t+1}) - x_hat, so the loss cannot shrink the error by shrinking the embeddings.
- Warm up lambda_err from 0 to 0.1 over the first 500 steps, and compare against feeding the predictor a detached state.
- Initialise h_0, M_0 and r_0 to zeros, and give the running surprise average a defined start (S_tilde = 1 at the first step).

Parallelism and training optimization

This subsection is a proposal. None of it is implemented or measured. Today the model processes one token at a time (Section 10), and the reference run took about 0.96 s per step at batch 8 x 32 on a CPU, with GPU speed unmeasured.

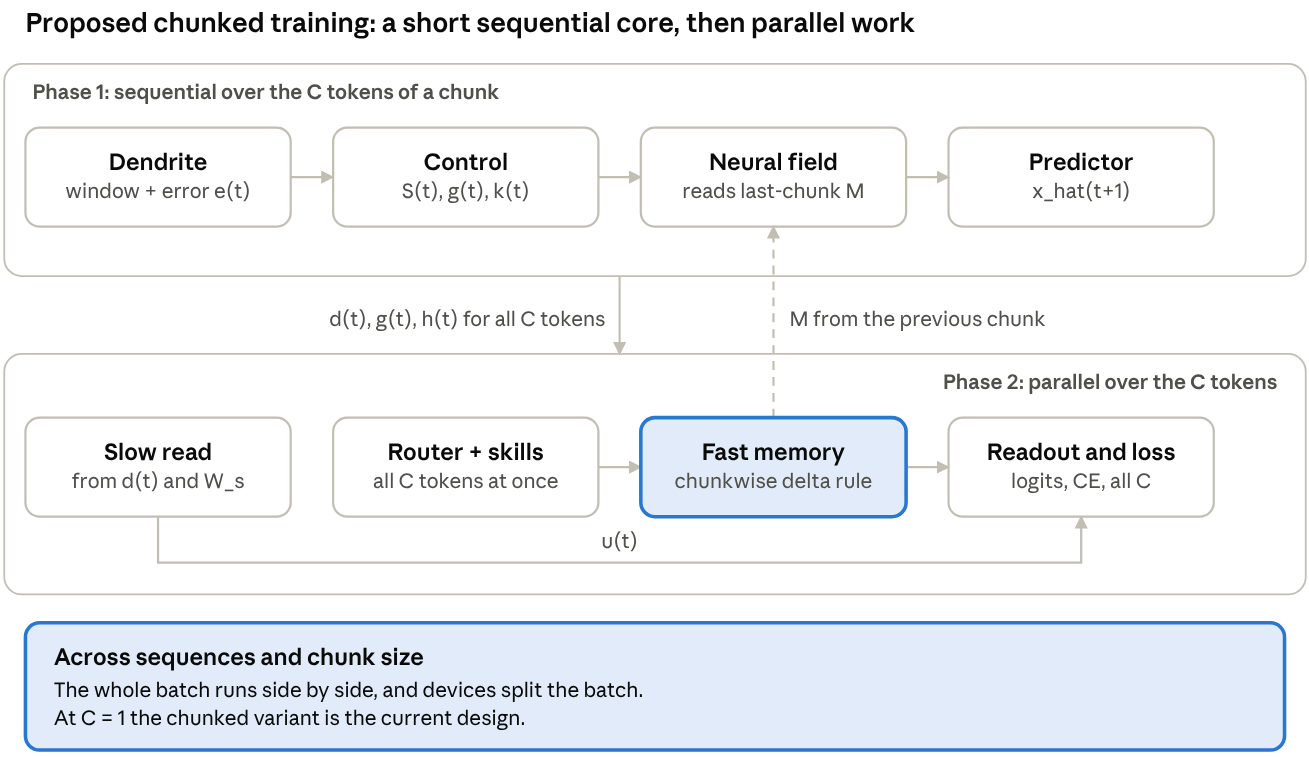
Why the core loop is sequential. In Appendix A, h(t) depends on h(t-1) and on the memory read r(t-1). The memory M(t-1) in turn depends on the skill output o(t-1), which comes from h(t-1). The field update is nonlinear (tanh and layer norm), so a parallel scan over time does not apply to the model as specified. This holds for the reference form; the parallel form of Section 3 removes the dependency on h(t-1).

What already runs in parallel. Sequences in a batch do not interact, because the fast memory is per sequence. The skills inside one token can also be computed side by side, since every skill is computed.

Fallback: chunked variant. If the whole-sequence parallel form of Section 3 fails its tests, split each sequence into chunks of C tokens and run each chunk in two phases.

- Phase 1, sequential: dendrite, control signals, neural field and predictor. The field has about 2.6K parameters in the reference model, so this loop is light.
- Phase 2, parallel: router and skills, fast-memory write and read, slow read, readout, logits and loss, for all C tokens at once. The delta-rule write could then use a chunkwise algorithm like the one DeltaNet (Yang et al., 2024) uses for the delta rule.

The price is that inside a chunk the field reads the memory as it stood at the end of the previous chunk, not at the previous token. With C = 1 this is the current design. A larger C should make training faster but weakens retrieval inside the field. Whether the chunkwise algorithm extends to the diagonal decay Lambda and the gate g(t) of this model still has to be checked.



proposed chunked training · two phases

Phase 1 stays sequential and phase 2 runs over the whole chunk. The dashed arrow is the one link that changes: the field reads the memory from the previous chunk.

Training optimization (proposed, untested).

1. Bigger batches and data parallelism across devices. Time cannot be parallelized in the current design, so the batch dimension has to keep the GPU busy, and a batch of 8 x 32 is small.
2. Mixed precision: bf16 for matrix products, and fp32 for the fast memory, the layer-norm statistics and the surprise average. After the change, rerun the gradient check of Appendix B item 6.
3. Compile the per-token step (torch.compile or CUDA graphs) to cut the launch cost of many small operations. Measure on a GPU first to see whether launch cost is the bottleneck.
4. Adaptive k breaks batching. Run every token to k_max with a mask, or bucket tokens by k, and report the real FLOPs.
5. Activation checkpointing per window, to allow a longer truncation window for the fast-memory gradient at lower memory cost.
6. Gradient accumulation, to get a larger effective batch on small hardware.

7. Optimizer parameter groups: no weight decay on the slow memory W_s, the gate parameters, the decay parameters and the norms. AdamW, warmup, cosine decay and clipping stay as above. Learning rates per group are untested.

How these changes are judged. Report tokens per second, time per step, peak memory and FLOPs per token for each change, on the same hardware as the GRU and transformer baselines. A speed-up is accepted only if the loss and the recall accuracy stay within the seed range of the sequential reference. A change that alters the result is reported as a change to the architecture, not as an optimization.

5. Component-by-component audit

Only the fast memory has experimental support; every other block is either unproven or has an open design problem. The table comes from the reported runs and from reading the equations in Appendix A. It is a design-level audit: the implementation code was not reviewed, so each finding still has to be confirmed there.

Block	Status	Main problem	Fix to try	First test
Dendrite	Gradient nonzero (Run 6); supplies the memory key and query	The error term gamma*e enters the key, so one token can get different keys in different contexts; the leakage test is a design review, not a run	Run the leakage test; build the key from the window part only	gamma = 0 against gamma > 0; log the norm of d_t
Predictor	Gradient nonzero; no effect on recall	Embedding collapse if the target is not detached; random tokens give no surprise signal; start of the surprise average undefined	Detach the target; warm up lambda_err; define the start value	Surprise on ABCABC against random tokens; embedding norm per epoch

Block	Status	Main problem	Fix to try	First test
Neural field	Gradient nonzero; reads the memory	The slow-memory read never reaches the field; r_{t-1} arrives one step late; h_0 undefined	Document the initial state; ablate a slow-memory term in the field	Multi-hop lookup with fixed against adaptive k
Neuromodulator	Gate receives gradient because the write is in the graph	Few learnable parameters (a_s, a_c, b); the bias delta reuses the same surprise signal as the gate; the gate can saturate; one scalar per token	Initialise g near 0.5; clamp g; optional content-dependent gate	Histogram of g; noisy stream with few important tokens; learned g against fixed g
Router and skills	Straight-through gives gradient to all 4 skills	Top-k is dense compute, not sparse; the gradient is biased; collapse onto one skill; balance term on pi, not on the selection	Choose dense or sparse and report FLOPs; balance on selection fraction	Skill usage histogram; task needing copy, reverse and count
Fast memory	Recall 1.000 on 4 pairs; about 0.40 without it	Decay and delta rule are inconsistent; capacity is about d_k pairs; the read follows the write, so the new value can be read back; decay range unconstrained	Delta rule on Lambda*M; Lambda = sigmoid(theta); cap g and the memory norm	8 to 64 pairs with distractors; overwrite task; read-before-write variant

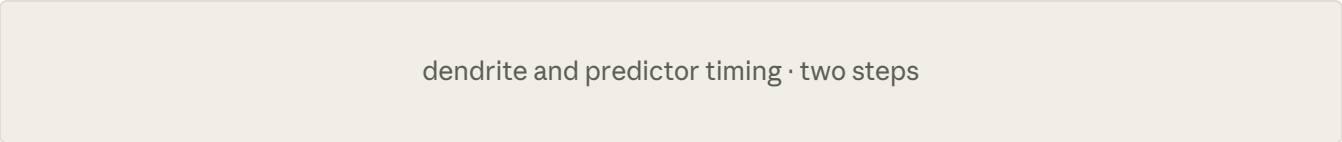
Block	Status	Main problem	Fix to try	First test
Slow memory and replay	Unproven; removing it did not change recall	Its read uses W′_q, the memory keys use W_k, so the spaces need not align; averaging unrelated sequences gives noise; two update rules act on W_s	Use W_q for the read; keep W_s out of weight decay; pick one update rule	Shared-facts task; continual learning, train A then B and measure A
Metacognition	Rules run; no evidence they help	Surprise is not difficulty; k is near constant on random data; round() has no gradient; the bias delta double counts surprise	Halt the field on convergence; fix one replay rule	Histogram of k; multi-hop task at equal FLOPs
Readout and logits head	Recall path works; order fixed from v1	Three terms of different scale in one sum; fixed mixing; memory can reach the logits through the field instead of the readout; vocabulary budget	Learned scalar per term; log each term′s norm; tie embeddings	Zero each term in turn; tied against untied embeddings

Every block that does not help on its own task is reported as "no evidence yet", not as useless. Metacognition is a rule-based controller and is described that way in this report.

Component diagrams

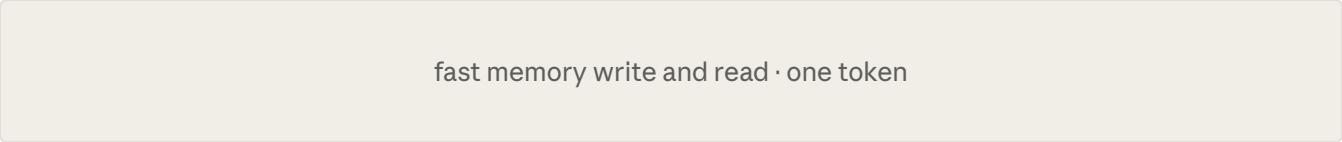
Four blocks are drawn in detail below: the dendrite with the predictor, the fast memory, the router, and the slow memory with replay.

Dendrite and predictor.



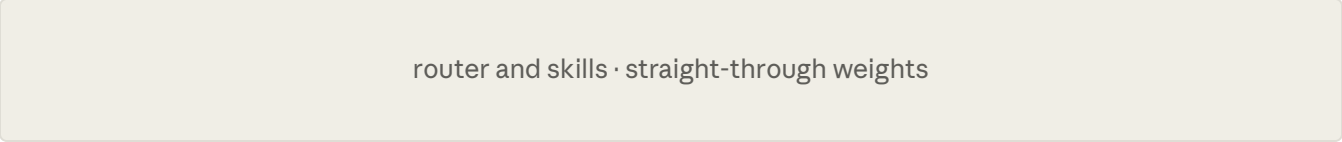
The predictor forecasts x(t+1) at step t; the error is formed only when x(t+1) arrives and enters the dendrite at step t+1.

Fast memory.



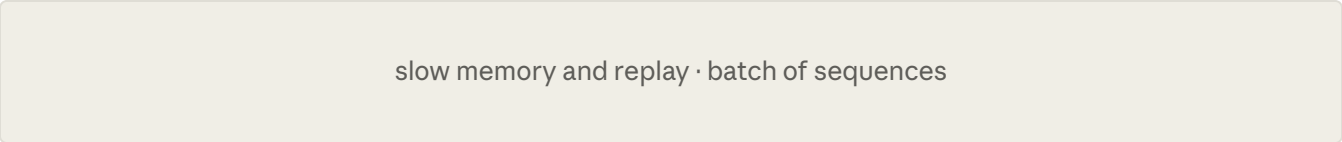
The write stores the difference between the new value and what the memory already predicts for that key, scaled by the gate. The dashed arrow is the control signal from the neuromodulator.

Router and skills.



The forward pass uses the top-k weights w, so unequal weights survive. The dashed arrows carry only the backward gradient of the full softmax. If every skill is still computed, as the audit suspects, the routing saves no compute (Section 5).

Slow memory and replay.



Every 50 to 100 steps the slow weights move 2% toward the surprise-weighted average of the fast memories in the batch. Backprop on the next-token loss also updates the same weights.

6. Parameter efficiency

In the first 10M implementation the embedding and logits head held 72% of the parameters, leaving about 2.8M for the architecture itself. A 10,000-token vocabulary at width 368 costs 3.68M in each of the two matrices.

Model	Total params	Largest block	Share
First implementation (vocab 10,000)	10.16M	Embedding + head	72%
Reference AESC (char vocab 65)	9.92M	Skills (8 operators)	85%

Two fixes apply. Tie the input and output embeddings, which saves about 3.7M at that size, or use a small vocabulary (characters, or 2,000 to 4,000 subword tokens). v1 said subword tokens balance capacity; the opposite holds, because a larger vocabulary inflates both matrices.

In the reference model the new blocks are small: the field has 2.6K parameters, the fast memory 0.33M and the slow memory 0.07M. Skills dominate, so a control with the same skills and none of the new blocks is needed before crediting the architecture for any gain.

Compute-matched protocol. Report parameters, FLOPs per token, training tokens and wall-clock time for every model. The k-step field multiplies the field's FLOPs by k, so compare at equal FLOPs as well as at equal parameters. Tied embeddings also feed the predictor target, so the embedding norm is logged to catch collapse.

7. Safety and monitoring

AESC exposes internal signals that a monitor can log at every token. These signals are tripwires, not proof that the model's behavior is safe. This section is a design and has not been implemented or evaluated.

Signals logged per token.

- Surprise S(t) and its running average
- Write gate g(t), write norm, and the norm of the fast memory
- Router choices and router entropy
- Field steps k(t)
- Consolidation events into slow memory

Risks from test-time plasticity. The fast memory changes while the model is used. Content written in one session can steer later outputs (memory poisoning), the memory can drift without bound, and state shared across users can leak information.

Controls.

1. Fresh state for every session, with no carry-over between users.

2. Caps on the write gate, the write norm and the memory norm.
3. An audit log of writes: token index, gate, key norm and value norm.
4. An anomaly detector that flags signals far from a baseline distribution, for example a z-score on S(t), g(t) and router entropy.
5. A freeze switch that sets g = 0, so the memory only decays (Proposition 4), with an optional reset.
6. Replay into the shared slow weights only from audited training sessions, never from live inference sessions, so a poisoned write cannot become permanent.

Evaluation plan. Inject adversarial text that tries to plant false associations. Measure the detection rate, the false-alarm rate, and whether a reset removes the effect.

Scope. A monitor on these signals can show that something changed, but not why, and it is not a faithful trace of the model's reasoning. Safety claims need red-team evaluation, which this report does not contain.

8. Experiments and evaluation plan

The reference implementation solves associative recall with fast memory on, the first implementation did not, and no multi-seed or multi-task comparison exists yet. All runs below are single-seed, small-configuration CPU runs. The recall task has 4 key-value pairs, 8 keys and 8 values, with the answer at the last position (chance 0.125).

#	Experiment	Model	Result
1	Recall, 800 steps	Reference AESC (d = 64)	Accuracy 1.000
2	Recall, fast memory removed, 300-step ablation	Reference AESC	Accuracy about 0.40
3	Recall, stop-gradient write, 800 steps	Reference AESC	Accuracy 0.75; gate and key-map gradient exactly 0
4	Recall, fast updates on or off, 500 steps	First implementation (d = 64, 4 skills)	Accuracy 0.37 to 0.39 either way

#	Experiment	Model	Result
5	Gradient audit, one backward pass	First implementation, 10.16M	Zero gradient in neuromodulator, metacognition, forgetting controller and field modulation input; router zero from the task loss
6	Gradient audit, one backward pass	Reference AESC, 9.92M	Nonzero gradient in all 11 blocks
7	Smoke run, char-level text, 200 steps, batch 8 x 32	AESC 9.92M, GRU 10.0M, Transformer about 10M	3.18, 2.91, 3.36 validation bits per character

Run 7 is too short to rank the models. Run 1 is consistent with the memory mechanism working on this task, but it is one seed on a toy task. Removing the slow memory, neuromodulator, metacognition, router or predictor did not change recall in the ablation, because this task does not need them, so there is no evidence yet that those blocks help.

Protocol for C2.

1. Tasks, one per block: recall with distractors and 8 to 64 pairs (fast memory); an overwrite task (decay); a noisy stream with few important tokens (neuromodulator); train task A then B and measure the drop on A (slow memory and replay); multi-hop lookup (field and metacognition); a task needing different operators (router); structured streams such as ABCABC against random tokens (predictor).
2. Models: the full AESC, a control with the same skills and none of the new blocks, one variant per removed block, a 10M transformer, and a 10M Mamba or GRU.
3. Same data, same training tokens, three or more seeds, mean and range reported, FLOPs per token listed.
4. A block counts as helping only if the full model beats the variant without it, outside the seed range, on the task built for that block. Otherwise the report says "no evidence yet".

Open items. Harder tasks that exercise the slow memory, neuromodulator and metacognition; the multi-seed runs above; GPU timing (the 10M model ran at about 0.96 s per step at batch 8 x 32 on CPU, and GPU speed is unmeasured); and the safety evaluation in Section 7.

Equation tests (not the model)

These tests check the algebra of the memory rule with random keys and values in numpy. They test the equations of this report, not the implementation, and they say nothing about learned behavior. They cover the memory part of the equivalence test of Section 3 only.

#	Test	Result
E1	The decay-consistent delta rule equals its linear form (I - g k k^T) Lambda M + g k v^T, with a diagonal Lambda	Max error 7e-16
E2	Chunkwise computation (a triangular solve per chunk) equals the sequential recurrence, no decay	Error 1.5e-15 on M and 1.4e-15 on the reads
E3	The same with a scalar decay of 0.95 and chunks of 8, the decay absorbed into the values	Error 7e-16
E4	The same in float32, scalar decay from 0.95 down to 0.3, chunks up to 64	Reads differ from a float64 sequential reference by at most 3.4e-7
E5	Write then read, and overwrite then read, for one key with g = 1	Error 2e-16 and 3e-16
E6	Recall of the rule alone, random unit keys, d_k = 64, 8 candidate values, for 4, 8, 16, 32, 64, 128 and 256 stored pairs	1 head: 1.00, 1.00, 1.00, 0.96, 0.78, 0.54, 0.33. 4 heads: 1.00, 1.00, 1.00, 1.00, 0.94, 0.74, 0.41 (chance 0.125)

E1 to E5 support the claim that the write is linear in M and that it can be computed in chunks for a scalar decay. The chunkwise computation with a diagonal Lambda was not tested. The decay-absorbing trick overflows once the decay to the power minus the chunk length leaves the float range, so long chunks or a strong decay need a numerically stable form. E6 shows that the rule alone, with random keys, holds recall at 1.00 up to 16 pairs at d_k = 64 and falls to 0.78 at 64 pairs. Four heads help, at four times the state, and the heads here use independent random keys. What learned keys do is untested.

9. Failure modes and fixes

Eleven failure modes are known. Rows 1 to 8 come from the audit of the first implementation, and rows 9 to 11 from review of v1 and follow-up tests.

#	Failure mode	Evidence	Fix
1	Fast weights averaged over the batch	One matrix shared by all sequences (code audit)	Per-sequence memory of shape batch x d_k x d_v
2	Writes frozen at inference	Update guarded by the training flag	Gated writes allowed at inference, scoped to the session and audited
3	Neuromodulator off the graph	Zero gradient (audit)	Gate multiplies the differentiable write
4	Dendrite as autoencoder	It predicted features of its own input	Predict the next input; error arrives one step late
5	Router weights ignored	Selected skills averaged equally; zero router gradient from the task loss	Weighted skills with straight-through gradient
6	Field without memory read	Modulation input unused, zero gradient	Memory read fed into the field every step
7	Forgetting controller unused	Never called; zero gradient; fixed decay 0.95	Learnable per-key decay inside the write
8	Vocabulary swallows the budget	72% of 10.16M parameters	Tied embeddings or smaller vocabulary
9	Stop-gradient write blocks learning	Recall 0.75 vs 1.000; gate gradient 0	Differentiable write in a truncated window
10	Possible label leakage	v1 equation e(t) = x(t+1) - pred (design review, not measured)	One-step delay; unit test in Appendix B
11	Figure 1 errors	Softmax before logits head, garbled label, unexplained N-times stack	Redrawn (Section 3)

10. Limitations and non-claims

No result here shows that a small AESC matches a larger model on any task. The evidence comes from small configurations, single seeds and one synthetic task.

- Only the fast memory is supported by an experiment. The slow memory, neuromodulator, metacognition, router and predictor are unproven (claim C2 is untested).
- The component audit in Section 5 is design-level: it was done from the report and its equations, not from the implementation code.
- The model processes tokens one at a time, so training is slower than a transformer of equal size; GPU throughput is unmeasured. The parallel form (Section 3) and the optimization ideas (Section 4) are proposals and have not been tested, and the reference results do not carry over to the parallel form.
- Metacognition is a set of simple rules, not a learned policy.
- The safety layer is a design, not a tested system.
- The author does not claim biological fidelity, convergence, unique skills or superiority to transformers.

11. Future work

The main proposal is a per-user context memory that keeps a long chat from being lost, without writing one user's chat into the shared slow weights. Everything in this section is a proposal and has not been tested.

Per-user context memory C_u. The fast memory M loses old context through decay, and the slow memory W_s is shared by all users. A third memory C_u, private to one user, sits between them:

```
C_u \leftarrow (1-\eta)\, C_u + \eta \sum_b \frac{S^{(b)}}{\sum_{b'} S^{(b')}}\, M^{(b)}
```

- During a chat, M writes as before. At the end of each turn, the important part of M is consolidated into C_u, with a norm cap and an audit log.
- A new turn or session starts from M_0 = C_u, or adds a read term P_u C_u^T q to the readout.
- C_u is isolated by user ID, and the user can reset it.

- W_s is never written from a live chat. This keeps the rule of Section 7 and adds one exception: live consolidation is allowed only into the user's private C_u.
- Test: state facts in session 1 and ask about them in session 2, with C_u on and off, over three or more seeds.

Other items from the audit.

- A slow-memory term in the field update, ablated against the current design.
- Convergence-based halting for the field steps in place of the rounded surprise rule.
- A content-dependent write gate.
- A learned controller to replace the Metacognition rules.

Appendix A. Forward equations

Symbols: x_t is the input embedding, h the field state, M_t the fast memory, and sg the stop-gradient operator. The equations in this appendix match the reference implementation; proposed changes are listed separately in Appendix A.2.

Dendrite and prediction.

```
d_t = \tanh\Big(\sum_{j=0}^{w-1} c_j \odot x_{t-j} + b\Big) + \gamma \odot
e_t
```

```
\hat{x}_{t+1} = f_p(h_t), \qquad e_{t+1} = x_{t+1} - \hat{x}_{t+1}, \qquad
S_{t+1} = \mathrm{sg}\big(\mathrm{mean}\,|e_{t+1}|\big)
```

Field (repeated k_t times). R is a shared local convolution over neighboring neurons.

```
h \leftarrow (1-\zeta)\odot h + \zeta \odot \mathrm{LN}\Big(\alpha \odot h +
\beta \odot \big(d_t + R(\tanh h) + W_m r_{t-1}\big)\Big)
```

Control. The running average of surprise is S_bar, and the relative surprise is S_tilde_t = S_t / S_bar. The gate bias is delta_t = +0.5 if S_tilde_t > 1.5, -0.5 if S_tilde_t < 0.5, and 0 otherwise.

```
g_t = \sigma\big(a_s(\tilde S_t - 1) + a_c\,\overline{|h_t|} + b +
\delta_t\big)
```

```
k_t = 1 + \mathrm{round}\Big((k_{max}-1)\,\mathrm{clip}\big(\tfrac{\tilde S_t
- 0.5}{1.5},\,0,\,1\big)\Big)
```

Router and skills.

$$\pi = \mathrm{softmax}(W_r h_t), \quad w = \mathrm{softmax}(\text{top-}k\ \text{logits}), \quad w^{STE} = w + \pi - \mathrm{sg}(\pi), \quad o_t = \sum_i w^{STE}_i\,\mathrm{Op}_i(h_t)$$

Fast memory. Lambda is a diagonal decay over key dimensions.

$$k_t = \frac{W_k d_{t-1}}{\lVert W_k d_{t-1} \rVert}, \qquad v_t = W_v o_t, \qquad M_t = \Lambda M_{t-1} + g_t\, k_t \big(v_t - M_{t-1}^{\top} k_t\big)^{\top}$$

Read and output.

$$q_t = \frac{W_q d_t}{\lVert W_q d_t \rVert}, \quad r_t = M_t^{\top} q_t, \quad u_t = (W'_q d_t)^{\top} W_s, \quad z_t = W_o\,\mathrm{LN}\big(o_t + P_f r_t + P_s u_t\big)$$

Replay into slow memory.

$$W_s \leftarrow (1-\eta)\,W_s + \eta \sum_b \frac{S^{(b)}}{\sum_{b'} S^{(b')}}\, M_t^{(b)}$$

Appendix A.2. Proposed forward changes (untested)

These changes come from the component audit in Section 5. They are applied one at a time, each with its own test, and none of them is part of the reference implementation or its results. The dendrite window feature is written d^win_t = tanh(sum_j c_j * x_{t-j} + b), so that d_t = d^win_t + gamma * e_t.

Change 1. Detached prediction target. The error loss can otherwise shrink the embeddings instead of improving the prediction. This replaces e_{t+1} = x_{t+1} - x_hat_{t+1}.

$$e_{t+1} = \mathrm{sg}(x_{t+1}) - \hat{x}_{t+1}$$

Test: the embedding norm does not fall over training, and recall is unchanged.

Change 2. Surprise average with a defined start. rho = 0.99 is a starting value to tune.

$$\bar S_t = \rho\,\bar S_{t-1} + (1-\rho)\,S_t, \qquad \tilde S_t = \frac{S_t}{\bar S_t + \epsilon}, \qquad \tilde S_1 = 1$$

Test: no division by zero, and the relative surprise falls on a repeating pattern such as ABCABC.

Change 3. Gate without double counting, with a cap. The bias delta_t is dropped, because it reuses the same surprise signal as the gate; dropping the a_s term instead is run as an ablation. The state h_t is the state after the k_t field steps of token t, and b starts at 0.

```
g_t = g_{max}\,\sigma\big(a_s(\tilde S_t - 1) + a_c\,\overline{|h_t|} +
b\big)
```

Test: the histogram of g is not stuck at 0 or at g_max, and learned g is compared with fixed g = 1.

Change 4. Field halting on convergence. This replaces the rounded rule for k_t. The field stops when the state stops changing, or at k_max.

```
k_t = \min\Big(k_{max},\ \min\{\, j \ge 1 : \lVert h^{(j)} - h^{(j-1)} \rVert
\le \epsilon \,\}\Big)
```

Together with Change 3, the surprise rules for k_t and delta_t are gone, and Metacognition only schedules replay. The rounded rule stays as the reference variant. Test: the histogram of k_t varies, and a multi-hop task is run at equal FLOPs.

Change 5. Delta rule consistent with the decay, and a bounded memory. The error is now computed against the decayed memory, so the decay and the write agree. Lambda is kept in (0, 1).

```
M^{\Lambda}_{t-1} = \Lambda M_{t-1}, \quad M_t = M^{\Lambda}_{t-1} + g_t\,
k_t \big(v_t - (M^{\Lambda}_{t-1})^{\top} k_t\big)^{\top}, \quad \Lambda =
\mathrm{diag}(\sigma(\theta))
```

```
M_t \leftarrow M_t \cdot \min\big(1,\ M_{max} / \lVert M_t \rVert_F\big)
```

Test: recall with 8 to 64 pairs, an overwrite task, and the memory norm logged per token.

Change 6. Key and query from the window feature only. The error term no longer changes the key of the same token in a different context.

```
k_t = \frac{W_k d^{win}_{t-1}}{\lVert W_k d^{win}_{t-1} \rVert}, \qquad q_t =
\frac{W_q d^{win}_t}{\lVert W_q d^{win}_t \rVert}
```

Test: gamma = 0 against gamma > 0, with recall compared.

Change 7. Slow read in the same query space. This replaces u_t = (W′_q d_t)^T W_s, so that the slow read and the fast memory use the same query.

```
u_t = q_t^{\top} W_s
```

Test: a shared-facts task, with the slow read on and off.

Change 8. Learned scale for each readout term. The three terms can have different scales, and the layer norm hides which one dominates.

```
z_t = W_o\,\mathrm{LN}\big(o_t + \beta_f P_f r_t + \beta_u P_s u_t\big),
\qquad \beta_f = \beta_u = 1 \ \text{at the start}
```

Test: each term is zeroed in turn, and the norm of each term is logged.

Change 9. Defined initial state.

```
h_0 = 0, \quad M_0 = 0, \quad r_0 = 0, \quad d_0 = 0, \quad e_1 = 0
```

Change 10. Loss: error warmup and a balance term on the selection. The error weight ramps up over the first 500 steps. B is computed from the fraction f_i of tokens routed to skill i and the mean router probability P_i, for n skills.

```
L = \sum_t \mathrm{CE}(z_t, y_t) + \lambda_{err}(s)\,\lVert e_t \rVert^2 +
\lambda_{bal}\, B, \qquad \lambda_{err}(s) = 0.1\,\min(1, s/500), \qquad B =
n \sum_{i=1}^{n} f_i P_i
```

Test: the usage histogram of skills, with no skill above 50% of the tokens.

Order of testing. Apply the changes in the order above. If recall on 4 pairs falls below 0.95 after a change, revert it and record the result in the report.

Appendix A.3. Parallel-form equations (proposed, untested)

These equations describe the parallel form of Section 3. The time index t runs over all T positions at once, and the pass index j runs over depth. They use the detached target, the causal surprise average and the decay-consistent delta rule of Appendix A.2. Change 4 (halting on convergence) is not used, because k is fixed per batch.

Start state.

$$H^{(0)}_t = \mathrm{Field0}\big(d^{win}_t\big), \qquad r^{(j)}_0 = 0, \qquad M^{(j)}_0 = 0$$

Pass j = 1, ..., k, for all t at once. The error and the dendrite output use the previous pass.

$$E^{(j)}_{t+1} = \mathrm{sg}(x_{t+1}) - f_p\big(H^{(j-1)}_t\big), \qquad E^{(j)}_1 = 0, \qquad D^{(j)}_t = d^{win}_t + \gamma \odot E^{(j)}_t$$

The surprise average is a causal moving average, which is a linear scan over t.

$$S_t = \mathrm{sg}\big(\mathrm{mean}\,|E^{(j)}_t|\big), \qquad \bar S_t = (1-\rho)\sum_{s \le t} \rho^{\,t-s} S_s, \qquad \tilde S_t = \frac{S_t}{\bar S_t + \epsilon}$$

$$g_t = g_{max}\,\sigma\big(a_s(\tilde S_t - 1) + a_c\,\overline{|H^{(j-1)}_t|} + b\big)$$

Keys, values and queries come from the previous position, the previous pass and the current position.

$$k_t = \frac{W_k D^{(j)}_{t-1}}{\lVert W_k D^{(j)}_{t-1} \rVert}, \qquad v_t = W_v\,\mathrm{RouterSkills}\big(H^{(j-1)}_t\big), \qquad q_t = \frac{W_q D^{(j)}_t}{\lVert W_q D^{(j)}_t \rVert}$$

The memory is linear in M for fixed k, v and g, so it can be computed for all t by a chunkwise algorithm.

$$M^{(j)}_t = \big(I - g_t k_t k_t^{\top}\big)\,\Lambda\, M^{(j)}_{t-1} + g_t\, k_t v_t^{\top}, \qquad r^{(j)}_t = \big(M^{(j)}_t\big)^{\top} q_t$$

$$H^{(j)}_t = (1-\zeta)\odot H^{(j-1)}_t + \zeta \odot \mathrm{LN}\Big(\alpha \odot H^{(j-1)}_t + \beta \odot \big(D^{(j)}_t + R(\tanh H^{(j-1)}_t) + W_m r^{(j)}_{t-1}\big)\Big)$$

Output and loss, for all t at once.

$$o_t = \mathrm{RouterSkills}\big(H^{(k)}_t\big), \qquad u_t = q_t^{\top} W_s, \qquad z_t = W_o\,\mathrm{LN}\big(o_t + \beta_f P_f r^{(k)}_t + \beta_u P_s u_t\big)$$

```
L = \sum_{t=1}^{T}\Big(\mathrm{CE}(z_t, y_t) + \lambda_{err}(s)\,\lVert
E^{(k)}_t \rVert^2\Big) + \lambda_{bal}\, B
```

Inference. The same equations run one token at a time. The state carried between tokens, for each pass j, is M^(j), H^(j) of the previous token, r^(j), D^(j) of the previous token, and the surprise average. Each pass keeps its own memory, so the state is k matrices instead of one, although the weights are shared across passes.

Appendix A.4. Proposed fixes to the parallel form (untested)

These equations change the memory part of A.3 according to the fixes in Section 3. If the fixes pass their tests, they replace the memory-per-pass part of A.3. The memory has n_h heads. The head index is left out: each head has its own keys, values, queries, scalar decay and matrix, and the reads of all heads are concatenated.

Context channel (fix 2). A diagonal linear recurrence over time, computed with a parallel scan.

```
c_t = a \odot c_{t-1} + (1-a) \odot W_c\, d^{win}_t, \qquad a =
\sigma(\theta_a), \qquad c_0 = 0
```

```
H^{(0)}_t = \mathrm{Field0}\big(d^{win}_t,\ c_t\big)
```

Single write pass (fix 1). Keys, values and gates are computed once, from the start state. The gate g_t is the one of A.3 with H^(0)_t in place of H^(j-1)_t.

```
k_t = \frac{W_k D^{(1)}_{t-1}}{\lVert W_k D^{(1)}_{t-1} \rVert}, \qquad v_t =
W_v\,\mathrm{RouterSkills}\big(H^{(0)}_t\big)
```

Scalar decay (fix 4). This is the rule of Change 5 with Lambda = lambda * I.

```
M_t = \lambda\,\big(I - g_t k_t k_t^{\top}\big) M_{t-1} + g_t\, k_t
v_t^{\top}, \qquad \lambda = \sigma(\theta_\lambda) \in (0,1), \qquad M_0 = 0
```

Reads in every pass (fix 1). The field step is the one of A.3 with r^(j)_{t-1}. The skills run again only at the end, o_t = RouterSkills(H^(k)_t), with the readout of A.3.

```
r^{(j)}_t = M_t^{\top} q^{(j)}_t, \qquad q^{(j)}_t = \frac{W_q D^{(j)}_t}
{\lVert W_q D^{(j)}_t \rVert}, \qquad j = 1, \dots, k
```

Heads (fix 5).

```
r^{(j)}_t = \big[\, r^{(j,1)}_t \,;\, \dots \,;\, r^{(j,n_h)}_t \,\big]
```

State at inference. One matrix per head, the context vector c_t, and for each pass the field state H^(j) and the dendrite output D^(j) of the previous token. This is n_h matrices instead of k.

Appendix B. Design checklist

1. Per-sequence fast state of shape batch x d_k x d_v.
2. The gate multiplies the write; writes are allowed at inference and are audited.
3. Router weights are used in the forward pass, and every skill receives a straight-through gradient.
4. The dendrite predicts the next input, and the error is computed one step late.
5. The field reads the memory at every step.
6. The neuromodulator and gate receive gradient: after one backward pass on the next-token loss alone, their gradient norm is above zero.
7. The recall test passes (accuracy of at least 0.95 on 4-pair recall) before any other claim is made.
8. Leakage test: change x(t+1) and check that the logits at step t do not change.
9. The stop-gradient ablation is reported next to the differentiable write.
10. A per-block parameter report is included, with embedding and head at 25% of the total or less (a target), or tied.
11. Baselines are compared at equal FLOPs as well as equal parameters.
12. The safety log fields are present: S(t), g(t), router ids, k(t), the memory norm and the write audit.
13. Each row of the audit table in Section 5 has a measured result, or is marked "no evidence yet".
14. The embedding norm is logged per epoch, so a collapse caused by the error loss is visible.

Appendix C. Propositions

These follow from the chain rule and the construction. They are not convergence results.

Proposition 1 (reachability). If a module's output enters the logits or the error through a differentiable map with a nonzero Jacobian, the next-token loss gives that module a gradient that can be nonzero. Evidence: Run 6 shows nonzero gradient in all 11 blocks of the reference model.

Proposition 2 (straight-through routing). Let w be the softmax over the top-k logits and pi the full softmax. Then w + pi - sg(pi) equals w in the forward pass, and its gradient is the gradient of pi. Every skill therefore receives a router gradient. This replaces the uniform 1/k forward weights of v1, which cannot express unequal skill weights.

Proposition 3 (revised: stop-gradient writes block learning). If the write M_t = Lambda M_{t-1} + g k (v - M^T k)^T is computed under stop-gradient, then M_t is constant with respect to every parameter, and W_k, W_v and the gate parameters receive no gradient through M. Observation: in Run 3 the gradient was exactly 0 and recall fell from 1.000 to 0.75. v1 claimed the stop-gradient design was justified; that claim is withdrawn.

Proposition 4 (zero gate). If g_t = 0 then M_t = Lambda M_{t-1}, so the memory only decays.

Proposition 5 (inference plasticity). With fixed parameters and g_t > 0, M changes across tokens during inference. This is why Section 7 requires per-session state and a write audit.

Proposition 6 (error supervision). The term lambda_err times the squared norm of e(t+1) sends gradient to the predictor and, through the state, to the field and dendrite. Surprise S is detached, so it carries no gradient.

Proposition 7 (no leakage). If the logits z_t depend on the inputs only through x(1..t), and errors are computed with a one-step delay, then the derivative of z_t with respect to x(t+1) is zero. The reference implementation processes one token per step, so causality should hold by construction; the Appendix B test has not been run yet.